

LAB 10

Title: Study and Implementation of Abstract Class in Java using Real-World Scenarios

Aim

To understand the necessity of abstraction by comparing program behavior **with and without abstract classes**.

Objectives

- To analyze limitations of normal class design
 - To implement abstract classes in Java
 - To observe correctness and flexibility improvements
 - To model real-world problems using abstraction
-

Theory

In object-oriented programming, using a normal class with common method implementation may lead to **generic or incorrect behavior** when subclasses require different logic.

An **abstract class** allows:

- Defining a common structure
- Forcing subclasses to implement specific behavior
- Preventing creation of incomplete or meaningless objects

Thus, abstraction improves **design correctness, safety, and scalability**.

Exercise 1: Payment Processing System

Part A: Without Abstract Class

```
class Payment {  
    double amount;  
  
    void pay() {  
        System.out.println("Processing payment of " + amount);  
    }  
}  
  
class UPI extends Payment {  
    String upId;
```

```

    UPI(double amount, String upiId) {
        this.amount = amount;
        this.upiId = upiId;
    }
}

class CreditCard extends Payment {
    String cardNumber;

    CreditCard(double amount, String cardNumber) {
        this.amount = amount;
        this.cardNumber = cardNumber;
    }
}

```

Part B: With Abstract Class

```

abstract class Payment {
    double amount;

    Payment(double amount) {
        this.amount = amount;
    }

    abstract void pay();
}

class UPI extends Payment {
    String upiId;

    UPI(double amount, String upiId) {
        super(amount);
        this.upiId = upiId;
    }

    void pay() {
        System.out.println("UPI Payment: " + amount + " using " + upiId);
    }
}

class CreditCard extends Payment {
    String cardNumber;

    CreditCard(double amount, String cardNumber) {
        super(amount);
        this.cardNumber = cardNumber;
    }
}

```

```
void pay() {  
    System.out.println("Card Payment: " + amount + " using " + cardNumber);  
}  
}
```

Exercise 2: Employee Salary System

Part A: Without Abstract Class

```
class Employee {  
    double salary;  
  
    void calculateSalary() {  
        System.out.println("Salary: " + salary);  
    }  
}
```

```
class FullTime extends Employee {  
    FullTime(double salary) {  
        this.salary = salary;  
    }  
}
```

```
class Intern extends Employee {  
    Intern(double stipend) {  
        this.salary = stipend;  
    }  
}
```

Part B: With Abstract Class

```
abstract class Employee {  
    double base;  
  
    Employee(double base) {  
        this.base = base;  
    }  
  
    abstract void calculateSalary();  
}
```

```
class FullTime extends Employee {  
    FullTime(double base) {  
        super(base);  
    }  
}
```

```

    void calculateSalary() {
        System.out.println("FullTime Salary: " + (base + 5000));
    }
}

class Intern extends Employee {
    Intern(double base) {
        super(base);
    }

    void calculateSalary() {
        System.out.println("Intern Stipend: " + base);
    }
}

```

Exercise 3: Notification System

Part A: Without Abstract Class

```

class Notification {
    void send() {
        System.out.println("Sending notification");
    }
}

class Email extends Notification {}
class SMS extends Notification {}
class Push extends Notification {}

```

Part B: With Abstract Class

```

abstract class Notification {
    abstract void send();
}

class Email extends Notification {
    void send() {
        System.out.println("Sending Email");
    }
}

class SMS extends Notification {
    void send() {
        System.out.println("Sending SMS");
    }
}

```

```
class Push extends Notification {  
    void send() {  
        System.out.println("Sending Push Notification");  
    }  
}
```

Exercise 4: Vehicle Fare System

Part A: Without Abstract Class

```
class Vehicle {  
    double distance;  
  
    void fare() {  
        System.out.println("Fare: " + distance * 10);  
    }  
}
```

```
class Car extends Vehicle {  
    Car(double d) {  
        this.distance = d;  
    }  
}
```

```
class Bike extends Vehicle {  
    Bike(double d) {  
        this.distance = d;  
    }  
}
```

Part B: With Abstract Class

```
abstract class Vehicle {  
    double distance;  
  
    Vehicle(double distance) {  
        this.distance = distance;  
    }  
  
    abstract void fare();  
}
```

```
class Car extends Vehicle {  
    Car(double d) {  
        super(d);  
    }  
}
```

```

    void fare() {
        System.out.println("Car Fare: " + distance * 15);
    }
}

class Bike extends Vehicle {
    Bike(double d) {
        super(d);
    }

    void fare() {
        System.out.println("Bike Fare: " + distance * 8);
    }
}

```

Observation

- Part A uses uniform fare logic
- Part B supports different pricing strategies

Analysis Questions

Common Questions

1. What is the main problem observed in Part A across all exercises?
2. How does an abstract class enforce correct implementation in Part B?
3. Why should the base class not have a fixed implementation for all subclasses?
4. What happens if a subclass does not implement an abstract method?
5. Which version is easier to extend when adding a new subclass? Justify.

Exercise-Based Questions

Payment System

1. Why is a generic pay() method not suitable for all payment types?

Employee System

2. Why must salary calculation differ between FullTime and Intern?

Notification System

3. Why should each notification type have a different send() implementation?

Vehicle System

4. Why is a single fare calculation incorrect in real-world transport systems?